Product Requirements Document

Image Forensics Workbench (IFW)

Version: 1.0
Status: Draft
Audience: Engineering Team

---

1. Overview

1.1 Product Summary

Image Forensics Workbench (IFW) is a local, browser-based image forensics application for security researchers and forensic analysts. It provides a professional-grade, configurable toolkit for deep multi-dimensional image analysis — covering file structure, metadata, compression artifacts, noise patterns, frequency domain, and more — with no data ever leaving the user's machine.

1.2 Design Philosophy

- **Local-first, always.** No server, no upload, no telemetry. All computation runs in the browser.
- **Analyst control.** Every tool exposes its parameters. Nothing runs as a black box.
- **Speed where possible, depth when needed.** Tools run on demand. Parameters trade off accuracy for performance explicitly.
- **Evidence-grade output.** Every result is exportable in a format appropriate to its data type.

---

2. Target Users

Primary: Security researchers and digital forensic analysts
Secondary: Academic researchers in media forensics, technical journalists performing image fact-checking

User expectations:

- Comfortable with technical parameters (e.g. block size, sigma values, DCT thresholds)
- Need reproducible, exportable results suitable for reporting
- Work with potentially sensitive images — privacy and custody of evidence is critical
- May work with large images (20MP+), RAW-adjacent formats, or unusual file structures

---

3. Platform & Technical Constraints

Constraint	Requirement
Execution environment	Browser (Chrome/Firefox/Edge, latest 2 versions)
Server dependency	None — fully offline capable after initial load
Data privacy	Zero network requests after app load. No external CDN calls during use.
CPU-only	No GPU/WebGL compute required. All heavy processing via Web Workers + WASM.
Image input	File System Access API (drag-and-drop and file picker)
Max image size	Graceful handling up to 50MP; configurable downsampling for expensive algorithms

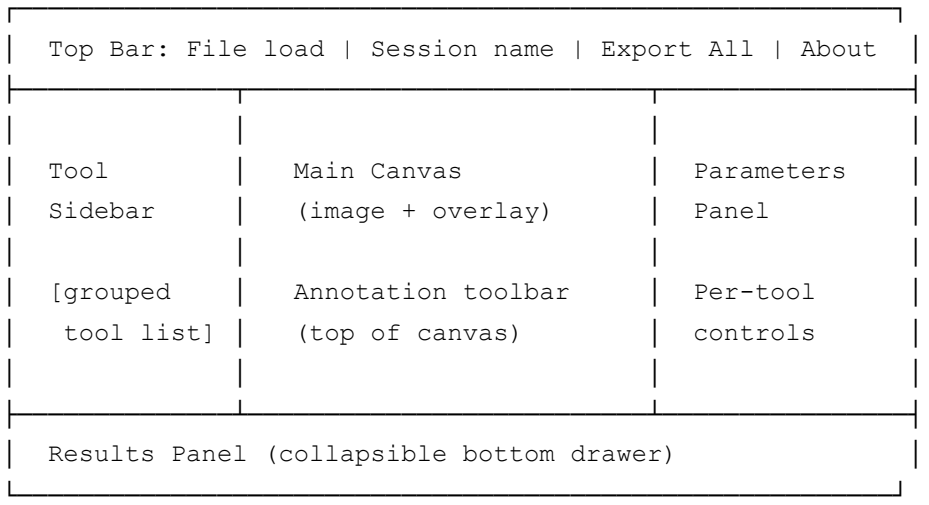
Key libraries (recommended):

- `OpenCV.js` (WASM) — image processing, clone detection, frequency transforms
- `exifr` — metadata extraction
- `fft.js` — Fast Fourier Transform
- `ONNX Runtime Web` (CPU backend) — AI-based forgery detection model
- `jsPDF` + `html2canvas` — PDF report generation
- `Fabric.js` or `Konva.js` — annotation layer
- `Comlink` — typed Web Worker interface

4. UI Architecture

4.1 Layout

The interface follows a **Photoshop-style single-page layout**:



4.2 Tool Sidebar

- Tools grouped by category (see Section 6)
- Each tool shows: icon, name, and a status badge (idle / running / complete / error)
- Clicking a tool activates it — loads its Parameters Panel and shows its last result on the canvas
- Tools can be pinned to a “Favorites” section at the top

4.3 Main Canvas

- Displays the original image by default
- When a tool is active, overlays its visual result on top (blended, with opacity slider)
- Supports zoom (scroll), pan (drag), and full-screen mode
- **Annotation Toolbar** (above canvas, visible when a tool result is active):
  - Draw rectangle, ellipse, freehand, arrow
  - Add text label
  - Color picker and stroke width

- Undo/redo annotations
- Clear annotations
- Export annotated canvas as PNG

4.4 Parameters Panel (right sidebar)

- Appears when a tool is selected
- All tool-specific controls rendered here (sliders, dropdowns, numeric inputs, checkboxes)
- **Run button** at the top — tools never auto-run; analyst always triggers execution manually
- **Progress indicator** during execution (spinner or progress bar where estimable)
- **"Reset to defaults"** button
- Execution time shown after run completes

4.5 Results Panel (bottom drawer)

- Collapsible drawer; expands when a tool finishes
- Shows: tool name, timestamp, execution time, result summary (text/stats)
- **Export button** per result (format is tool-appropriate — see Section 6)
- History: last N results per tool are retained in-session

---

5. Core Application Features

5.1 Image Loading

- Drag-and-drop onto canvas
- File picker button (File System Access API, no copy made to browser storage)
- Supported input formats: JPEG, PNG, WebP, TIFF, BMP, GIF (first frame), HEIC (with fallback)
- On load: display image info bar (filename, dimensions, file size, color space, bit depth)
- Raw bytes are retained in memory alongside the decoded pixel data for file-structure tools

5.2 Session Management

- A session = one loaded image + all tool results + annotations within that session
- Sessions are held **in memory only** — no persistence to disk unless user explicitly exports
- "New Session" clears everything after a confirmation prompt

5.3 Export — Global

- **Export All Results** in top bar: packages all completed tool results into a ZIP archive
  - Each tool result exported in its own appropriate format (see tool specs)
  - Includes a `manifest.json` with tool name, parameters used, and timestamp per result
- **PDF Report**: generates a paginated PDF with: image thumbnail, all result images, annotations, parameter tables, and result summaries

---

6. Analysis Tools

Each tool specification follows this format:
Purpose / Parameters / Output / Export format / Performance notes

---

6.1 Category: File & Metadata

6.1.1 Metadata Viewer

Purpose: Extract and display all embedded metadata from the image file.
Parameters: None (display only)
Output:

- EXIF fields (camera make/model, lens, focal length, aperture, shutter speed, ISO, timestamps)
- GPS coordinates with map link (opens external map in new tab — only user-initiated)
- IPTC fields (copyright, creator, keywords)
- XMP fields (editing software, history, color profile)
- Embedded thumbnail (rendered separately, dimensions noted)

- Makernote fields (raw hex + decoded where known)

Export: JSON (full structured dump) + TXT (human-readable summary)

6.1.2 File Structure Inspector

Purpose: Examine raw file bytes, detect appended data, validate file signatures.
Parameters:

- Byte range start/end for hex view
- Bytes per row (8 / 16 / 32)

Output:

- File signature validation (magic bytes vs. declared format)
- Hex dump view with ASCII sidebar
- Chunk/segment map for JPEG (SOI, APP0–APP15, DQT, SOF, SOS, EOI) and PNG (IHDR, tEXt, zTXt, IDAT, IEND etc.)
- Detection of data appended after EOF marker
- File entropy visualization (bar chart per 1KB block — high entropy may indicate encrypted/compressed appended data)

Export: TXT (hex dump), JSON (chunk map + anomalies)

6.2 Category: Compression Analysis

6.2.1 JPEG Quantization Table Analyzer

Purpose: Extract and evaluate JPEG quantization tables to fingerprint compression history.
Parameters: None
Output:

- Raw quantization table values (luminance + chrominance) as grid
- Quality factor estimate (derived from table values)
- Comparison against known encoder profiles (Photoshop, Lightroom, iOS, Android, social media platforms)

- Double-compression indicator: detects ghost quantization artifacts suggesting the image was previously compressed at a different quality level

Export: JSON (raw tables + analysis), PNG (table heatmaps)
Note: Requires custom JS JPEG parser (browser decoder discards quantization data)

6.2.2 Block Artifact Visualizer

Purpose: Visualize JPEG 8×8 DCT block boundaries and their artifact strength.
Parameters:

- Enhancement factor (1–10)
- Grid overlay toggle

Output:

- Block boundary map overlaid on canvas (heatmap where strong boundaries = potential re-compression or manipulation)
- Regions with abnormally strong or absent block artifacts are highlighted

Export: PNG (annotated overlay), JSON (per-block artifact scores)

6.3 Category: Error Level Analysis

6.3.1 ELA (Error Level Analysis)

Purpose: Re-compress the image at a known quality and amplify the difference to reveal regions inconsistent with the rest of the image — a common manipulation indicator.
Parameters:

- Re-save quality (1–95, default 75)
- Amplification factor (1–20, default 10)
- Scale mode: per-channel or luminance only
- High-pass pre-filter toggle (reduces uniform-region noise)

Output:

- ELA difference image displayed as heatmap overlay on canvas

- Histogram of ELA values across the image
- Highlight regions above a configurable ELA threshold

Export: PNG (ELA heatmap), JSON (parameters + per-region statistics)

6.3.2 Multi-Quality ELA

Purpose: Run ELA at multiple quality levels simultaneously to distinguish genuine artifacts from incidental ones.
Parameters:

- Quality levels to compare (multi-select: 50, 65, 75, 85, 95)
- Amplification factor

Output:

- Grid of ELA results per quality level, side-by-side
- Difference map between quality levels (helps isolate real anomalies)

Export: PNG (composite grid)

6.4 Category: Noise Analysis

6.4.1 Noise Map Visualizer

Purpose: Extract and visualize the noise residual layer of the image, which reflects sensor characteristics.
Parameters:

- Denoising method (Gaussian, Median, BM3D-lite)
- Kernel/sigma size (3–21)
- Channel (All / R / G / B / Luminance)
- Amplification factor

Output:

- Noise residual map overlaid on canvas
- Noise inconsistency heatmap — regions where noise texture differs significantly from

the rest may indicate splicing or compositing

Export: PNG (noise map), JSON (per-region noise statistics: mean, variance, MAD)

6.4.2 PRNU (Photo Response Non-Uniformity) Analysis

Purpose: Extract the sensor fingerprint from the image for source camera identification or consistency checking.
Parameters:

- Wavelet filter type (Daubechies / Symlet)
- Number of wavelet levels (1–4)

Output:

- Estimated PRNU noise pattern (displayed as normalized image)
- Per-region PRNU correlation map (highlight regions that don't match the dominant sensor pattern)

Export: PNG (PRNU pattern), JSON (correlation scores)
Note: Full PRNU matching requires a reference camera fingerprint. In v1, only intra-image consistency is analyzed.

6.5 Category: Frequency Domain

6.5.1 FFT Spectrum Analyzer

Purpose: Visualize the frequency spectrum of the image to detect periodic patterns, re-sampling artifacts, and compositing traces.
Parameters:

- Channel (Luminance / R / G / B)
- Window function (Hanning / Hamming / Blackman / None)
- Log scale toggle
- Spectrum zoom level

Output:

- 2D FFT magnitude spectrum displayed on canvas (log-scaled for visibility)

- Detected periodic peaks highlighted (grid pattern from re-sampling is a classic forgery indicator)
- 1D radial frequency profile chart

Export: PNG (spectrum image), JSON (peak locations + magnitudes), CSV (radial profile data)

6.5.2 DCT Coefficient Viewer

Purpose: Inspect DCT coefficient distributions, useful for detecting double compression and statistical anomalies.
Parameters:

- DCT component (AC1–AC63, DC)
- Block grid region (full image or user-selected ROI)
- Histogram bin count

Output:

- Histogram of selected DCT coefficient values across all 8×8 blocks
- Anomalous periodicity in histograms (double-compression artifact) flagged automatically

Export: PNG (histogram), CSV (raw coefficient values), JSON (analysis summary)

6.6 Category: Clone & Copy-Move Detection

6.6.1 Keypoint-Based Clone Detector

Purpose: Detect copy-moved or cloned regions within the image using feature point matching.
Parameters:

- Algorithm (ORB / AKAZE — both CPU-only via OpenCV.js)
- Max keypoints (500 / 1000 / 2000 / 5000)
- Match threshold (distance ratio, 0.5–0.9)
- Min cluster size (minimum matched keypoints to constitute a detection)

- Pre-processing downscale factor (1x / 0.75x / 0.5x — for performance)

Output:

- Matched keypoint pairs drawn on canvas with connecting lines
- Detected cloned region pairs highlighted with bounding rectangles
- Match count and confidence score per detection

Export: PNG (annotated result), JSON (match pairs + coordinates)
Performance note: Run at 0.5x scale for images >5MP unless accuracy is critical

---

6.6.2 Block-Matching Clone Detector

Purpose: Dense block-based approach — more thorough than keypoints but slower. Better for textured or low-detail regions.
Parameters:

- Block size (8 / 16 / 32 px)
- Step size (overlap control: 1–16 px)
- Similarity threshold
- Min clone region area (px²)
- Downscale factor

Output:

- Duplicate block map overlaid on canvas
- Clustered clone region pairs highlighted

Export: PNG (annotated result), JSON (duplicate block coordinates)

6.7 Category: Color & Lighting

6.7.1 Channel Separator & Analyzer

Purpose: Split and independently analyze image color channels to detect compositing inconsistencies.
Parameters:

- Color space (RGB / HSV / LAB / YCbCr)
- Display mode (side-by-side / single channel fullscreen)

Output:

- Individual channel images rendered on canvas
- Per-channel histogram
- Per-channel statistics: mean, std deviation, min, max, entropy

Export: PNG per channel, JSON (statistics), CSV (histogram data)

6.7.2 Lighting Direction Estimator

Purpose: Estimate light source direction per region to detect compositing of elements lit from different directions.
Parameters:

- Grid size (number of regions: 4×4 / 8×8 / 16×16)
- Estimation method (gradient-based / specular highlight based)

Output:

- Arrow overlay per grid cell showing estimated light direction
- Inconsistency heatmap (cells where direction significantly deviates from the dominant direction)

Export: PNG (annotated overlay), JSON (per-cell direction vectors + deviation scores)

6.7.3 Histogram Analyzer

Purpose: Deep histogram analysis across channels and color spaces.
Parameters:

- Color space (RGB / HSV / LAB)
- Bin count (64 / 128 / 256)
- Region of interest (full image or canvas selection)

Output:

- Per-channel histogram chart (interactive, zoomable)
- Anomaly detection: comb effect (sign of levels adjustment), clipping, unusual gaps

Export: PNG (chart), CSV (bin values), JSON (detected anomalies)

6.8 Category: Steganography Detection

6.8.1 LSB Plane Visualizer

Purpose: Visualize individual bit planes to detect LSB steganography — hidden data embedded in the least significant bits.
Parameters:

- Channel (R / G / B / All)
- Bit plane (0 = LSB to 7 = MSB)

Output:

- Selected bit plane rendered as binary image on canvas
- LSB plane entropy score (high entropy in LSB may indicate steganographic content)
- Visual noise pattern analysis (structured patterns in LSB = potential steganography)

Export: PNG (bit plane image), JSON (entropy scores per plane)

6.8.2 Statistical Steganography Tests

Purpose: Run statistical tests for common steganography techniques.
Parameters:

- Tests to run (multi-select): Chi-Square, RS Analysis, Sample Pairs
- Channel selection

Output:

- Per-test result with p-value or estimated hidden data percentage
- Verdict per test: Likely clean / Inconclusive / Likely contains hidden data
- Chart showing test statistic vs. expected baseline

Export: JSON (test results + statistics), TXT (plain-language summary)

6.9 Category: AI-Based Detection

6.9.1 AI Forgery Detector

Purpose: Run a pre-trained CNN model (ONNX, CPU backend) to produce a per-region forgery probability map.
Parameters:

- Model selection (bundled models — e.g. MantraNet-lite, CAT-Net-lite, or similar open-weight models)
- Tile size (256 / 512 px)
- Stride (overlap between tiles)
- Confidence threshold for highlighting

Output:

- Heatmap overlay on canvas showing per-region forgery probability
- Global score: probability the image has been manipulated
- Per-region bounding boxes above threshold

Export: PNG (heatmap), JSON (per-tile scores + global score)
Note: Models are loaded lazily on first use. Model file sizes must be ≤50MB to keep the app practical. Community-contributed models can be added via a model registry JSON file.

6.10 Category: Raw Inspection

6.10.1 Hex / Binary Viewer

Purpose: Inspect raw file bytes directly.
Parameters:

- Byte offset and length
- Bytes per row
- Encoding display (Hex / Decimal / Binary / ASCII)

- Search: byte pattern or ASCII string

Output:

- Hex dump view with highlighted search matches
- Jump-to-offset navigation
- Bookmarked offsets (analyst can add labels)

Export: TXT (hex dump), JSON (bookmarks + annotations)

6.10.2 String Extractor

Purpose: Extract printable strings from the raw file, useful for finding embedded URLs, software names, or hidden text.
Parameters:

- Minimum string length (4–32)
- Encoding (ASCII / UTF-8 / UTF-16)
- Filter by type: URLs, file paths, software strings, all

Export: TXT (string list), JSON (strings with byte offsets)

---

7. Annotation Layer

Available on the Main Canvas whenever a tool result is active.

Tools:

- Rectangle, ellipse, freehand pen, arrow, line
- Text label (with font size and color)
- Measurement ruler (px distance between two points, with scale calibration)

Controls:

- Color picker + opacity
- Stroke width
- Fill toggle

Behavior:

- Annotations are per-tool-result — switching tools shows that tool's annotations
- Undo / redo stack (min. 50 steps)
- Annotations are included in PNG exports and PDF reports
- Annotations stored in session memory as vector data (not burned into the result image until export)

---

8. Export System

Per-Tool Export

Each tool's export button produces the most relevant format for its output type:

Data type	Format
Visual result / heatmap / overlay	PNG
Numerical data / coordinates / scores	JSON
Tabular / series data	CSV
Plain text summary / hex dump	TXT
Metadata summary	JSON + TXT

Global Export (Export All)

- ZIP archive containing all completed tool exports
- `manifest.json` at root with: session timestamp, image filename + hash (SHA-256), tool list with parameters and execution time per tool
- Optional: full PDF report (paginated, includes canvas screenshots with annotations, parameter tables, text summaries)

Image Hashing

On load, compute and display SHA-256 and MD5 of the original file. These are included in all exports to support chain-of-custody documentation.

---

9. Performance Requirements

Scenario	Target
App initial load	< 5 seconds (including OpenCV.js WASM)
Metadata extraction	< 500ms
ELA (10MP image)	< 3 seconds
FFT Spectrum (10MP, luminance)	< 2 seconds
Clone detection ORB (10MP, 1000 kp, 0.5x scale)	< 15 seconds
AI forgery detection (10MP, 256px tiles)	< 60 seconds
UI responsiveness during computation	Main thread never blocked — all computation in Web Workers

All computationally expensive tools must:

1. Run exclusively in Web Workers
2. Support cancellation (abort controller pattern)
3. Report progress (0–100%) back to the UI during execution

---

10. Error Handling

- Unsupported file format: clear error message with supported format list
- File too large (>200MB): warn user and offer to proceed with downsampling
- Tool execution failure: show error in results panel with technical detail (collapsible) and option to retry
- WASM/ONNX load failure: graceful degradation — affected tools marked as unavailable with explanation
- All tool failures are non-fatal — other tools remain fully operational

---

11. Accessibility & UX Standards

- Keyboard navigation for all tool sidebar items
- High-contrast mode support
- Canvas zoom via scroll wheel and keyboard shortcuts (`+` / `-` / `0` to reset)
- Tool shortcuts (configurable in settings)
- Dark theme default (appropriate for forensic/analyst workstation environments)
- All charts and visualizations must include a text data table alternative

---

12. Out of Scope for v1

- Multi-image comparison (compare two images side by side across tools)
- Video frame analysis
- Network-based model updates
- User accounts or cloud sync
- Collaboration / multi-user sessions
- Mobile browser support
- Plugin/extension API

---

13. Success Metrics (v1 Launch)

Metric	Target
All 18 tools functional and exportable	100%
Zero network requests after initial load	Verified by network audit
No data written to browser storage	Verified (no localStorage, IndexedDB, cookies)
Core tools complete on 10MP image within targets	Performance test suite pass

Annotation layer usable on all visual tools	100% coverage

---

14. Suggested Tech Stack Summary

Layer	Technology
Framework	React + TypeScript
Build	Vite (fast WASM asset handling)
Image processing	OpenCV.js (WASM)
Metadata	exifr
FFT	fft.js
AI inference	ONNX Runtime Web (CPU)
Annotation canvas	Fabric.js or Konva.js
Charts	Recharts or D3.js
PDF export	jsPDF + html2canvas
ZIP export	JSZip
Worker bridge	Comlink
Styling	Tailwind CSS